Manuscript
Click here to access/download;Manuscript;ElasticActor-IJPP.tex
Click here to view linked References

Noname manuscript No.
(will be inserted by the editor)

ElasticActor: An Actor System with Automatic Granularity Adjustment

Peng Zhao · Lei Liu · Wei Cao · Xiao Dong · Jiansong Li · Xiaobing Feng

Received: date / Accepted: date

Abstract The actor model has gained significant popularity in recent years, because its high-level abstraction facilitates the construction of massive concurrent distributed systems, e.g. cloud applications. However, building an efficient actor system is a challenging task. In particular, the concurrency granularity has a critical impact on the performance of actor systems. Moreover, the best granularity depends on both the available hardware and the characteristics of the workload.

This paper proposes a new actor system, ElasticActor, that automatically adjusts the granularity of parallel execution. In ElasticActor, all actors are assigned to a number of actor stations. Actors in the same station share a mailbox and constitute the minimum scheduling unit. Therefore, the runtime system can adjust the granularity by changing the number of actor stations in the system. More specifically, we model the average latency and derive a closed-form formula to compute the best number of stations. The experimental results show that ElasticActor outperforms two state-of-the-art actor systems, CAF and Orleans. It gains up to 1.3× throughput and decreases the average latency by 0.75× thanks to the granularity adjustment.

Keywords Actor model · Concurrency granularity · Cloud computing · Performance optimization

Peng Zhao · Lei Liu · Wei Cao · Xiao Dong · Jiansong Li · Xiaobing Feng
State Key Laboratory of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China
E-mail: {zhaopeng, liulei, caowei, dongxiao, lijiansong, fxb}@ict.ac.cn

Peng Zhao · Xiao Dong · Jiansong Li
School of Computer and Control Engineering, University of Chinese Academy of Sciences, Beijing 100049, China

1 Introduction

As cloud computing continues to evolve, actor model has attracted increasing attention for its ease of building interactive distributed cloud applications. The actor model, originally proposed by Hewitt [6] in 1973, populates the system with a large amount of isolated concurrent computing units (i.e. actors), which interact with each other through messages. Note the facts that the advance of multi-core and many-core processor architecture enables massive concurrency, and the advent of cloud computing and big data demands a programming language or library providing good performance through concurrency with high productivity. These facts revived interest in the actor model. Though highly parallel, the actor model can still improve on productivity, especially when used for demanding systems such as cloud computing and big data applications. As a result, much research has been carried out to devise better actor systems, from early actor languages like Erlang [1], to recent actor systems such as CAF [2], Akka [3] and Orleans [4]. These actor systems make it easy to build complicated interactive cloud applications [25, 26]. However, it remains difficult for actor systems to sustain high throughput while ensuring low service latency.

We discover that the concurrency granularity is a critical factor in actor system performance, and the current design doesn't coordinate well with the computing resources and the characteristics of workloads. Most existing implementations of the actor model treat an actor as the elementary work item and create a number of scheduling threads to select these items dynamically for execution. The benefit of this design is that the fine-grained concurrency provides good scalability. However, the actor scheduling and switching overheads decrease the performance. Typical applications involve millions of actors, and these actors are switched frequently by the scheduling threads because the execution time of handling a message is usually short. In a simple application, our experiments show that the overheads of actor management increase the average request latency by 1.54× for CAF and 25.15× for Orleans (see Sect. 2.2).

Therefore, reducing the overhead of the actor management is critical to improving the performance of actor applications. Unfortunately, it is hard in existing actor systems since the granularity in these systems is fixed. The only solution is manual, letting application programmers handle the overhead problem by increasing the granularity and creating fewer "bigger" actors. For example, a programmer can create an actor for each group of four actors rather than for each player, reducing the number of player actors to a quarter. Application programmers have to handle the overhead problem by creating "bigger" actors which handle messages of several actors to decrease the number of actors in system. However, this approach is not satisfying for three reasons: 1) Programmers need to write more code which is irrelevant to the application logic, thereby making the program more complex and harder to understand and maintain; 2) It reduces the scalability of the actor system. Since the number of actors cannot be changed dynamically when additional machines are

added to the system. 3) The greatest problem is that it is hard for programmers to find the optimal number of actors to achieve the best performance. We will show later that the best number of actors depends on both the hardware resources and the characteristics of the workload. Neither information is readily available to the programmer, making it impossible for programmers to determine the optimal granularity.

In summary, the problems we face to reduce the overhead of actor management are: 1) How to make the optimization transparent to application programmers; 2) How to improve the performance without sacrificing scalability; 3) How to achieve the best performance in every different scenario. To solve the first problem, we need to make application-independent optimizations for actor systems. The second problem is hard because high-level granularity conflicts with scalability. The last problem is also challenging since there are many factors that impact.

We propose ElasticActor in this paper, which is a new design alternative to actor systems. We define a new concept named an *actor station*, which is a collection of actors and the unit of scheduling. All actors in a station share a mailbox. Each actor created in the system is assigned to a station. Since the number of actors in the system is fixed at any certain point in time, if we create more stations in the system, we reduce the number of actors in each station and consequently the granularity of parallel execution.

The runtime system can adjust the granularity periodically by changing the number of stations in the system. The number is computed by collecting relevant information such as the number of working threads and message arrival rate of actors. The ElasticActor runtime can adjust the granularity, for example, to reduce the granularity to preserve scalability.

For the optimal granularity problem, we formulate a mathematical model of the average request latency of the system. We use a layered queuing network (LQN) to model the actor system. A closed-form solution is obtained to guide the system find the optimal granularity.

We evaluate ElasticActor with several applications. ElasticActor outperforms two state-of-the-art actor systems, CAF and Orleans. It gains up to 1.3× throughput and decreases the average latency by 0.75× thanks to the granularity adjustment. Our contributions are as follows:

1. We discover that the overhead of work item scheduling in actor systems is not trivial. The optimal granularity depends on both hardware availability and workload characteristics.
2. We propose a new actor system, ElasticActor, which enables the adjustment of granularity with *actor stations*.
3. We formulate a model of the average request latency of the system and conclude an analytical closed-form solution to find the optimal granularity.
4. We implement and evaluate the proposed actor system. The experimental results show significant improvements on both latency and throughput in real-world applications compared with two existing actor systems.

2 Granularity in Actor Systems

The actor model adopts the philosophy that *every task is an actor*, an actor is the basic unit of concurrent computation with isolated private state. Actors communicate with others through messages. Upon receiving a message, an actor can concurrently change its private state, send messages to other actors, create more actors and designate the actions to take when it receives the next message.

2.1 Granularity in existing actor systems

Actor-level granularity is a common choice in existing actor systems (CAF, Orleans, Erlang, etc.). In these systems, an actor is the smallest execution unit. Actors with pending messages are dispatched to a thread pool. Every scheduling thread in the pool repeatedly selects a ready actor, retrieves and handles the messages in its mailbox.

However, the overhead of actor scheduling and context switching undermines overall system performance when dealing with a very large amount of actors. Unfortunately, this is the common case: most actor-based applications typically consist of millions of actors or even more [10].

2.2 Concurrency granularity impacts

To reveal the performance impact of concurrency granularity in actor systems, we devise a micro-benchmark, vector addition, in Orleans (Experiment 1).[1]

There is only one type of actors in the vector addition micro-benchmark, computation actors. The vectors to be added are divided into sub-vectors evenly by a client. The client then sends messages containing the ranges of the sub-vectors to the actor system. With a fixed workload (vector length), we manually adjust the concurrency granularity by changing the number of actors.

In all the following experiments, the vector length is 2^{32} and the length of the sub-vectors is 2^{10}, which means there are 2^{22} messages sent by the client. Fig. 1 presents the average response time under varied concurrent granularities.

Experiment 1 shows clearly that the concurrency granularity strongly influences the performance of the system. As the number of actors increases, the average latency gradually decreases to 4.7ms, because more actors help to improve the load balance of the system. However, after the number of actors increases to 64, the average latency starts to increase dramatically (up to 25.15× slower) because of the overhead from frequent actor creation and management. [2]

[1] The testing platform is described in Sect. 5.

[2] Similar phenomenon is observed in CAF. The average latency with default granularity is 1.54× longer than the one with the optimal granularity.

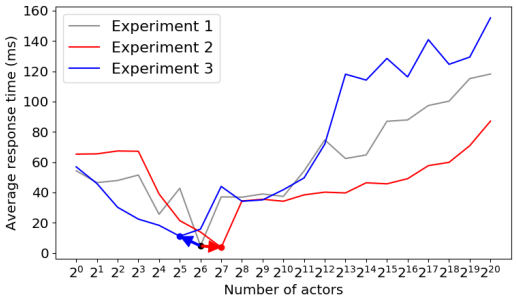


Fig. 1 The average response time under varying concurrency granularity in Orleans. The work in each experiment is fixed, and more actors means finer-grained granularity.

To further investigate the influence of workload characteristics and hardware resources on the selection of optimal granularity, we conduct two additional experiments based on Experiment 1.

We use a workload with better locality in Experiment 2, which means that the same actor is more frequently accessed in a short duration of time. We manage to do this by reordering the messages sent by the client. It reduces the overhead of actor management because there are less distinct actors within a time window. However, the loads among the actors are poorly balanced. Therefore, for a small number of actors (up to 16), load imbalance dominates and results in a longer response time compared to Experiment 1. But as the number of actors increases further, the response time becomes shorter than Experiment 1 as the locality reduces the actor-switch overhead. Hence, finer-grained concurrency is desirable as the red arrow indicates in Fig. 1.

In Experiment 3, we use half of the threads used in Experiment 1. In this case, the computing resource is constrained, and less parallelism is available, coarser-grained concurrency is preferable. The experimental results confirm our reasoning. Both workload characteristics and hardware resources impact on the optimal granularity.

3 ElasticActor

Adjusting the concurrency granularity can effectively improve the performance of actor systems. In this section, we propose ElasticActor, an actor system with automatic concurrency granularity. ElasticActor is composed of a language based on C++, a compiler and a runtime system as illustrated in Fig. 2. The rest of this section explains the key concepts and modules in ElasticActor.

3.1 ElasticActor Language

The ElasticActor language extends C++ by adding the actor support including the actor definition, message definition and message passing primitive.

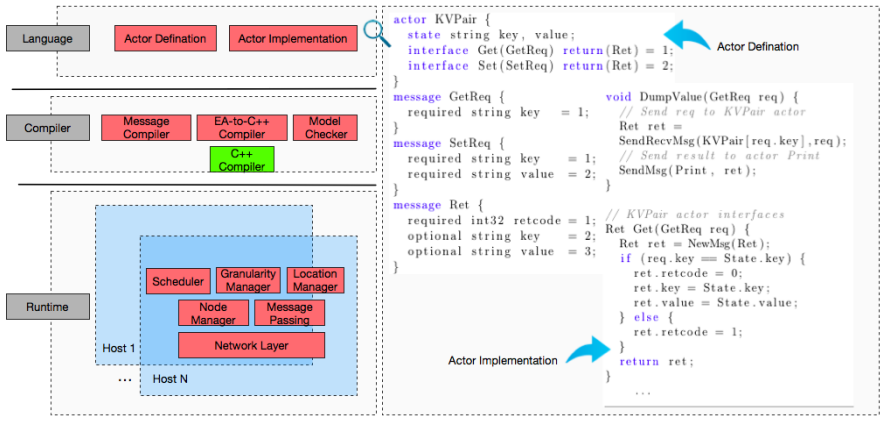


Fig. 2 An overview of ElasticActor

Actor definition Message type definition follows the same format in Protocol Buffers. Definition of an actor type includes state definition and interface definition. Fig. 2 shows an example actor definition where actor *KVPair* and relevant messages are defined. *KVPair* is a type of actor that maintains a key-value pair and provides access interfaces *Get* and *Set*.

Actor implementation There are two kinds of actor types in ElasticActor: stateful and stateless. Stateful actors have states and are identified by an ID. The ID can be either an integer or a string that programmers designate to an actor. There is no ID for stateless actors. Programmers can refer to a stateless actor by its actor type name. ElasticActor supports both asynchronized and synchronized message communication to make programming more flexible.

Fig. 2 shows an example actor implementation. On receiving a new *GetReq* message, the *DumpValue* actor sends the message to the *KVPair* actor with ID *key*, waits the reply and then sends the reply to a stateless actor *Print*.

3.2 Compiler

ElasticActor compiler is responsible for translating ElasticActor language to C++ and provides supports for debugging and optimization. As illustrated in Fig. 2, it includes four components: The message compiler generates message data structure definition and access interfaces in C++. EA-to-C++ compiler converts actor implementation code to C++ code. Programs that violates the constraints in ElasticActor are recognized by the model checker. The above three components are implemented with LLVM. The translated C++ code is then compiled to shared libraries with a commodity C++ compiler.

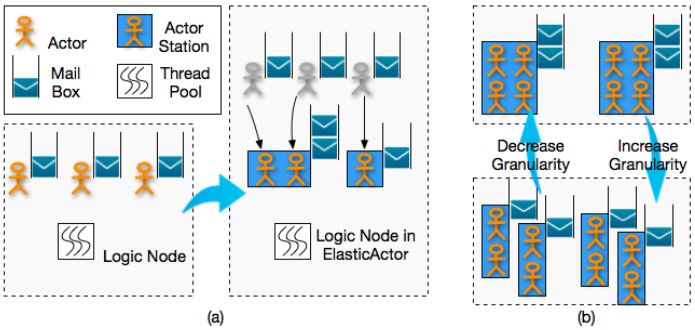


Fig. 3 (a) Actor stations in ElasticActor (b)Granularity adjustment

3.3 Runtime

ElasticActor can run on a cluster of servers, each running a host process to load the shared libraries of actors and run the application. The components such as scheduler and location manager have common design with existing actor systems. Here, we describe key concepts introduced in ElasticActor.

Actor station A new concept *actor station* (or *station* for brevity) is introduced. The station (see Fig. 3(a)) is the smallest execution unit and composed of a set of actors of the same type and a shared mailbox. All the messages sent to the actors in a station are queued in the mailbox. With actor stations, the granularity can be adjusted by merging and splitting stations. As illustrated in Fig. 3(b), since the number of actors in the system is fixed, more stations means fewer actors in each station, resulting in finer-grained concurrency.

Granularity adjustment The process of granularity adjustment is as follows: the runtime recomputes and changes the number of stations after a period of time. Some of the actors then are migrated to new stations to re-balance the stations. In the migration, the state and messages belonging to the actor are copied to the new station, the location directory is then updated. New messages to the actor will be sent to the new station. Messages arrived during the migration are kept and redirected to the new stations after the migration is done. All the strategies including adjustment timing, station and migrating actor selection are programmable by the internal ElasticActor interfaces.

It is worth mentioning that actor migration is not a significant cost in ElasticActor because the migration is not frequently triggered and it is efficient even if triggered. The granularity is adjusted with a new number of stations only if the results of recomputation within a certain time are around the new number within a certain deviation. The mechanism is adopted to avoid thrashing. Meanwhile, it makes migration happen infrequently. Besides that, previous studies [9, 24] show that actor migration is very efficient.

Table 1 Notations

Notation	Description
K	#Actor classes
$b_i^{(2)}$	Msg process time variance in class i
n_i	#Actor stations in class i
ρ_i	Load of class i
λ_i	Msg arrival rate to actor class i
r	Switch cost between actor stations
b_i	Mean msg process time in class i
ρ	Total load of the system

4 Granularity model

ElasticActor makes it possible to adjust granularity of the system automatically by changing the number of stations at runtime. To find the optimal granularity, we model the ElasticActor runtime system as a layered queuing network (LQN), and find a closed-form solution by solving the latency minimization problem.

4.1 Problem statement

To build a mathematical model, we next give a formal description of the actor system with notations and an optimization objective.

Formal description Table 1 gives a list of notations used in the mathematical model. Consider an actor application with K classes of actors, the system will create a set of stations for each class of actors. We use the notation n_i to indicate the number of stations for the i-th class. For the i-th actor class, the total arrival rate of messages is λ_i. The processing time of each message varies. The mean processing time of the messages of the actor class i is denoted by b_i, and the variance is denoted by $b_i^{(2)}$. Then $\rho_i = \lambda_i \times b_i$ is the load of the i-th actor class. The sum of loads of all actor classes, $\rho = \sum_1^K \rho_i$, is the total load of the system. After all messages in a station's mailbox are processed, the scheduler will choose another ready station to execute. Let r be the mean time needed by a scheduler to switch from one station to another. The switch cost is assumed to be independent from the arrival rate and message processing time.

4.2 Optimization model

The LQN is a natural fit for modeling the performance of the actor system. In the queuing theory, the basic queuing model consists of *queues*, at which *customers* arrive. The customers wait in the queue until they are served by the *server*. A queuing network consists of different kinds of queues. When a customer leaves from a queue, it will join another queue or leave the system with a

probability. LQN consists of multiple layers, where the servers of one layer act as the customers in the layer below. LQN is a well-studied model in the queuing theory. A large number of studies have developed the mathematical models of various types of LQNs and their applications (see surveys [13] and [14]). In these studies, LQN is extensively used in the performance evaluation and optimization of telecommunication systems, virtualization technologies, database management systems, and more. However, to the best of our knowledge, there is no study on improving the performance of an actor system by LQN.

Adapting LQN in ElasticActor We employ a special LQN model called the polling system. A polling system is a two-layer queuing network. There is one (or more) server(s) in the bottom layer. Servers in the bottom layer poll the queues in the top layer and serve the queued customers. In the actor system, which is the case we are interested in, messages are customers, and actors correspond to servers in the top layer of the polling system. For the bottom layer, actor stations are the customers and threads are servers. To avoid ambiguity, we use the terminology of LQN in the remaining part of this section.

Deriving the latency metric After modeling the actor system as a polling system, it is straightforward to see that the mean waiting time of a customer is the average latency we target. However, studies show that, for the polling system we used, the mean waiting time cannot be expressed with an expression. Instead, we use the weighted sum of the mean waiting time as our objective, which can be expressed with an expression using the pseudo-conservation law [15]:

$$\sum_{i=1}^{K} \rho_i W_i = \frac{\sum_{i=1}^{K} \rho_i b_i^{(2)} + R\rho}{2t(1-\rho)} \tag{1}$$

where W_i is the mean waiting time of messages in stations of class i, and $R = (\sum_1^K n_i)r$ is the total switch cost for the scheduler to switch all the stations one by one for a round. This formula has been used for optimizing peer-to-peer networks [16] and service-oriented systems [17].

Taking the load balance into consideration Equation 1 gives the weighted sum of the average waiting time. It captures the performance of an actor system with a single thread. Optimization for Equation 1 suggests fewer actor stations. However, actor systems are usually distributed and parallel. Fewer stations can lead to load imbalance. Load balance must be considered.

Let the random variables X_i be the amount of work dispatched to thread i, then the vector $(X_1, X_2, ..., X_t)$ follows multinomial distribution with the vector of means $E(X_1, X_2, ...X_t) = (np_1, np_2, ..., np_t)$ and covariance matrix $\underset{t\times t}{B} = (b_{ij})$, where $b_{ii} = np_iq_i,\ b_{ij} = -np_ip_j(i \neq j)$. Assuming that actor stations are dispatched to threads randomly with equal probability, we have $p_i = \frac{1}{t}$ and $q_i = 1 - p_i = 1 - \frac{1}{t}$ for all $i = 1, ...t$. For simplicity and traceability, we

incorporate the effect of load balance by adding the standard variance of the amount of work dispatched to a thread,

$$lb = \frac{\eta t}{K} \sum_{i=1}^{K} \frac{\rho_i}{n_i} \sqrt{n_i \frac{t-1}{t^2}} \tag{2}$$

where η is with the unit of $[time]$ in order to make the load balance measure compatible with the weighted sum of the mean waiting time. Its value is determined by solving the minimization problem of Formula 3 on representative tests with known optimal configurations, which are obtained off-line by manually changing the number of stations in the system. The rest of Equation 2 besides η is the standard variance of the amount of work in threads. We then conclude the optimization problem as follows:

$$\begin{aligned} &\underset{\{n_i\}}{minimize} \ \frac{\sum_{i=1}^{K} \rho_i b_i^{(2)} + R\rho}{2t(1-\rho)} + lb \\ &subject\ to\ \rho \leq t \end{aligned} \tag{3}$$

where the constraint ensures that a system is not over-loaded when it is in a stable state. This non-linear optimization problem can be solved:

$$n_i^{opt} = \sqrt[3]{(t-1)(\frac{\eta t \rho_i (1-\rho)}{K r \rho})^2} \ \ , \ for\ i = 1, ..., K$$

The solution is derived from the first order condition of optimality.

4.3 Estimating Parameters

The parameters required by the optimization model are t, λ_i, ρ, K, r. Some of the parameters are easy to obtain: the total number of scheduling threads t and the number of actor classes K are available values in the ElasticActor runtime. The arrival rates λ_i can be measured by counting the number of messages received by the stations of class i in a period of time. The mean message process time b_i and switch time r can also be deduced by averaging the recorded time. With the number of classes K, arrival rate λ_i and mean message process time b_i obtained, the total load ρ can be then computed.

5 Evaluation

In this section, we analyze the performance of ElasticActor, with three micro-benchmarks and a real-world application, and compare with CAF and Orleans. To better evaluate the effects of granularity adjustment, we also implemented

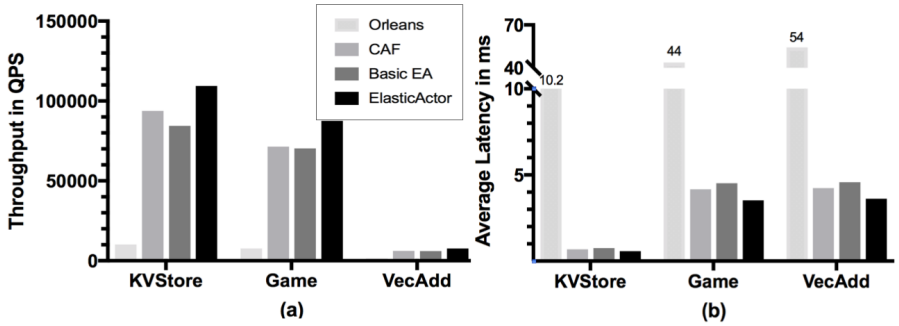


Fig. 4 Throughput (a) and latency (b) in three micro-benchmarks

another system, Basic EA, by eliminating the station and granularity adjustment mechanism from ElasticActor.

Testing platform The experimental testbed consists of four servers each with 8GB of memory and a dual-core Intel Core i5-4570 (3.2GHz) processor. An additional machine with 16GB of memory and a dual-core Intel Core i5-4590 (3.3GHz) processor is used as the client to generate workload. We use CAF v0.14.4 and Orleans v1.5.

5.1 Overall Performance

We implement three micro-benchmarks with four systems: Orleans, CAF, Basic EA and ElasticActor. The three benchmarks are KVStore, Game and VecAdd. KVStore is a in-memory key-value store system in which each actor stores a key-value pair. Game is a simplified game server which can be found at Orleans website as a demo. VecAdd is a vector addition application described in Sect. 2.2.

We evaluate the performance with two metrics: average latency and throughput. The throughput is measured by *request per second (QPS)*. We keep the test system fully loaded to maintain the peak throughput. After that, the client is set to send requests at the peak throughput to measure the average latency. By this testing strategy, we can avoid the inaccuracy of the measured latency caused by the system overload.

Results are illustrated in Fig. 4. We can see that although the performance of Basic EA is worse than CAF (0.95× in throughput and 1.09× in latency on average)[3], ElasticActor with granularity adjustment outperforms both CAF (1.21× in throughput and 0.85× in latency) and Orleans (10.84× in throughput and 0.07× in latency). Compared with Basic EA, the granularity adjustment gains throughput by 1.27× and decreases latency by 0.78×.

[3] The implementation of CAF is very efficient, and it outperforms many other actor systems such as Charm++ and Akka. [11]

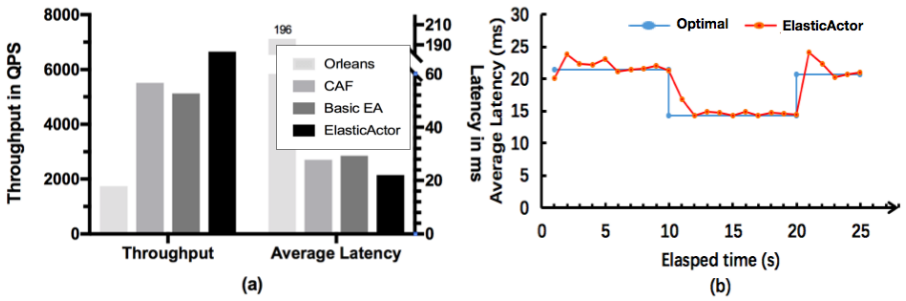


Fig. 5 (a)Throughput and latency in scale-out NAS (b) Latency with time evolving

5.2 A real-world application: scale-out NAS

We also study the performance of ElasticActor with a real-world application, scale-out network-attached storage (NAS). It is for massive data storage. It employs a fully symmetric distributed architecture, supports either file or object storage and provides super-large single file system for unstructured storage. Our implementations (including ElasticActor, Basic EA, CAF and Orleans) are derived from the software of a commodity scale-out NAS storage system named *OceanStor* [12] designed by Huawei. There are 8 classes and millions of actors.

Fig. 5(a) shows the throughput and average latency of the four different scale-out NAS implementations. ElasticActor has 7.45× QPS with 0.11× latency compared to Orleans and 1.21× QPS with 0.8× latency compared to CAF. ElasticActor still gets 1.3× throughput and 0.75× latency compared to Basic EA, thanks to the granularity adjustment.

It is not surprising that the performance of Orleans is far behind the other three systems as Orleans is written in C#. With the features such as JIT compilation and garbage collection, C# programs can be slower than C++ programs [5].

To test the effectiveness of our optimization model, we collect the average latency in every second in a 25-second run of the scale-out NAS. At first, the client sends requests at 6K QPS. After 10 seconds, we decrease the request speed by a half. After another 10 seconds, we shut down two servers. We also find the optimal latency in the three settings (four servers+6K, four servers+3K, two severs+3K) by manually adjusting the granularity. Fig. 5(b) shows the results. We can see that ElasticActor can find the optimal granularity under all the three settings. Besides, the overhead of actor migration is acceptable as analyzed in Sect. 3.3.

5.3 Mailbox performance

There are two major differences between the two systems, ElasticActor and Basic EA due to the elimination of the station. 1) *Message passing*: in Basic

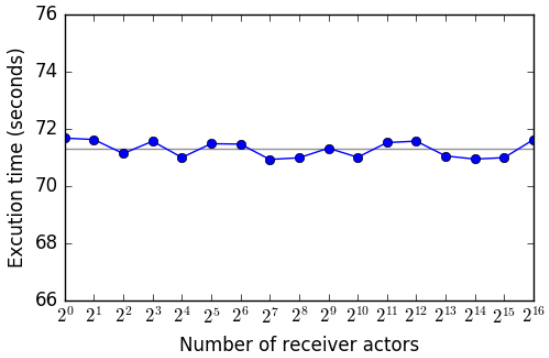


Fig. 6 Mailbox benchmark execution time with varying receivers

EA, each of the actors has a mailbox while all the actors in the same station share a mailbox in ElasticActor. 2) *Actor management*: the execution unit in Basic EA is an actor rather than a coarse-grained station in ElasticActor. We device a message passing benchmark to find out how much performance gain is achieved thanks to granularity adjustment and how the mailbox sharing affects the performance.

In the message passing benchmark, there are a total number of 65536 sender actors, and each of the senders sends 100 messages to a receiver actor. The receiver simply receives the message and does nothing else. The program terminates when all the receivers have received their messages.

We implement the benchmark with Basic EA. By changing the number of receivers, we simulate message passing scenarios in ElasticActor with different number of stations and mailboxes. For example, if the number of receivers is set to 2, half of the senders will send their messages to one receiver and the other half will send messages to the other receiver. This is equivalent to the case where there are two stations in ElasticActor, and each of them encapsulates half of the receiver actors.

We use Basic EA rather than ElasticActor in this benchmark because we need to wipe out the effect of the actor management factor. For all test cases, we create 65536 receiver actors no matter how many receivers are actually responsible to accept messages, eliminating the influence of actor creation overhead. Besides that, we employ the 1-gated service mechanism which means a scheduled actor must yield the processor after processing 1 message. This guarantees the number of actor switch times is all the same in all test cases. We measure the clock time, and measurements are averaged over 10 independent runs.

Figure 6 visualizes the time consumed by the applications to send and process 6553600 messages as a function of available number of receivers/mailboxes. As expected, the execution time doesn't vary with the the number of receivers/mailboxes. We can conclude that the station doesn't introduce message passing overhead thanks to the efficient lock-free mailbox data structure

we use in ElasticActor, which is first proposed in CAF [11]. The performance gain of ElasticActor comes mainly from the granularity adjustment mechanism.

6 Related Work

Optimization of actor system ActOp [9] is an application-independent runtime mechanism for optimizing service latency of actor-based applications. It automatically identifies frequently communicating actors and migrates them to the same server. The optimization in ActOp is orthogonal to granularity adjustment. Emilio et al. [10] optimized actor language Erlang for NUMA platform by providing interfaces for programmers to mark actors as hubs which frequently communicate with other actors. The runtime then groups the actors in the system with hubs as centers and dispatch the groups to different NUMA nodes. Karmani et al. [18] analyze and compare the performance of various actor-frameworks in JVM. It demonstrates that suitable optimizations can promote the efficiency with some useful communication and synchronization abstractions on the JVM platform. Salvaneschi et al. [19] introduce Context-oriented programming (COP), a language-level solution, to implement context-aware software, highlighting a large performance improvement in the actor model.

Granularity Adjustment Nathan et al. [8] built a tool to measure and analyze the performance of high-level multi-threaded program models. They use two metrics *idleness* and *overhead* (similar to load balance and actor-related overhead in this paper) to guide the programmers to increase or decrease concurrency. Gordon et al. [20] address the gap between the original granularity of the program and the underlying granularity of the architecture. They demonstrate a coarse-grained software pipelining to expose the data parallelism of the streaming applications. Yan et al. [21] present an algorithm to adjust parallel loop scheduling granularities to match the workload at runtime. They show that using runtime knowledge to dynamically adjust scheduling granularity is a practical way to process parallel loops without prior available knowledge of the execution. Acar et al. [22] propose techniques for controlling granularity implicitly from the aspect of decreasing the task-creation overheads. Dally [23] discusses that very fine granularity down to the level of instructions is required to extract parallelism for enabling the continued performance scaling in the field of exaScale computing.

7 Conclusion

In this paper, we show that concurrency granularity is critical to the overall performance on actor systems. We propose ElasticActor in which the concept actor station is introduced to enable automatic granularity adjustment. We

use LQN to model the execution of ElasticActor. A closed-form solution of the best station number is obtained. Evaluation with real-world applications shows the proposed granularity adjustment approach is effective.

Acknowledgements We would like to thank the anonymous reviewers for their comments and valuable feedback. Special thanks to Chen Ding for his comments and suggestions which have helped the development and presentation of the paper. The research is supported by the National Key R&D Program of China (Grant No.2016YFB1000402), National Natural Science Foundation of China (Grant No.61521092, No.61802368, No.61432018, No.61332009).

References

1. Erlang. http://www.erlang.org/
2. C++ Actor Framework. http://www.actor-framework.org/
3. AKKA. http://akka.io/
4. Orleans. https://www.microsoft.com/en-us/research/project/orleans-virtual-actors
5. Benchmark games. https://benchmarksgame.alioth.debian.org/
6. Hewitt, C., Peter, B., Richard, S: A universal modular ACTOR formalism for artificial intelligence. International Joint Conference on Artificial Intelligence (1973)
7. Bernstein, P., Bykov, S., Geller, A., Kliot, G., Thelin, J: Orleans: Distributed virtual actors for programmability and scalability. MSR-TR-2014-41 (2014)
8. Tallent, Mellor-Crummey: Effective Performance Measurement and Analysis of Multithreaded Applications. PPoPP'09, pages 229-240, New York, NY, USA (2009)
9. Newell, A., Kliot, G., Menache, I., Kliot, G., Menache, I., Gopalan, A., Akiyama, S., Silberstein, M: Optimizing distributed actor systems for dynamic interactive services. Eleventh European Conference on Computer Systems, EuroSys'16 (2016)
10. Francesquini, E., Goldman, A., Méhaut, J F: NUMA-Aware Runtime Environment for the Actor Model. 2013 42nd International Conference on Parallel Processing, ICPP'13, pages 250-259 (2013)
11. Charousset, D., Hiesgen, R., Schmidt, T C. Revisiting Actor Programming in C++. Computer Science, 45:105-131 (2015)
12. OceanStor. http://e.huawei.com/en/products/cloud-computing-dc/storage/massive-storage/9000
13. Vishnevskii, V., Semenova, O: Mathematical methods to study the polling systems. Automation and Remote Control, 173-220 (2006)
14. Chen, H., Yao, D: Fundamentals of queuing networks: Performance, asymptotics, and optimization. Springer Science and Business Media (2013)
15. Takagi, H: Analysis and application of polling models. Performance Evaluation: Origins and Directions. Springer Berlin Heidelberg, 423-442 (2000)
16. Sanghavi S., Hajek B., MASSOULIE L. Gossiping with multiple messages. IEEE Transactions on Information Theory, 4640-4654 (2007)
17. Tribastone M., Mayer P., Wirsing M. Performance prediction of service-oriented systems with layered queuing networks. International Symposium On Leveraging Applications of Formal Methods, Verification and Validation. Springer Berlin Heidelberg, 51-65 (2010)
18. Karmani, Rajesh K., Amin Shali, Gul Agha. Actor frameworks for the JVM platform: a comparative analysis. Proceedings of the 7th International Conference on Principles and Practice of Programming in Java. ACM (2009)
19. Salvaneschi, Guido, Carlo, G., Matteo, P: ContextErlang: introducing context-oriented programming in the actor model. Proceedings of the 11th annual International Conference on Aspect-oriented Software Development. ACM (2012)
20. Gordon, Michael I., William Thies, and Saman Amarasinghe. Exploiting coarse-grained task, data, and pipeline parallelism in stream programs. ACM SIGOPS Operating Systems Review, 151-162 (2006)
21. Yan Yong, . Adaptively scheduling parallel loops in distributed shared-memory systems. IEEE TPDS, 70-81 (1997)

22. Acar, U A., Chargueraud, A., Rainey, M: Oracle Scheduling: Controlling Granularity in Implicitly Parallel Languages. ACM Sigplan Notices, 46(10):499-518 (2011)
23. Dally, Bill. Power, programmability, and granularity: The challenges of exascale computing. 2011 IEEE International Test Conference. IEEE, (2011)
24. Desell, T., Varela, C A: A Performance and Scalability Analysis of Actor Message Passing and Migration in SALSA Lite. AGERE!'15 (2015)
25. Bernstein, P. A., Dashti, M., Kiefer, T., Maier, D: Indexing in an Actor-Oriented Database. In CIDR (2017)
26. Wu, C., Faleiro, J M., Lin, Y., Hellerstein, J M: Anna: A KVS For Any Scale. In 34th IEEE International Conference on Data Engineering (2018)